

Wskaźniki i funkcje (podprogramy), rekurencje, sortowanie tablic

- Przypisać zmiennej `int x` wartość `5`. Zadeklarować wskaźnik `int* px`, i przechować w nim adres zmiennej `x`.
 - wyświetlić na ekranie wartość `x` i jego adres, nie używając wskaźnika.
 - wyświetlić potem na ekranie wartość `x` i jego adres, ale z użyciem tylko wskaźnika.
 - zmienić wartość `x` używając wskaźnika. Wyświetlić wartość `x` po podmianie oraz jego adres, wskaźnikiem bądź `&`.
 - zadeklarować zmienną `int y`, przypisać wartość różną od `x`.
 - przypisać pod wskaźnik `px` teraz adres zmiennej `y`
 - wyświetlić wartość i adres zmiennej `y`, używając wskaźnika `px`, sprawdzić też, że będzie taka sama jak byśmy użyli `y` i `&y`.
- Napisać funkcję `double to_minutes(double seconds)`, która przekonwertuje wartość sekund w argumencie `seconds` na minuty i zwróci wynik jako `double`.
- Napisać funkcję `void to_minutes_inplace(double* ps)`, która będzie w stanie podmienić wartość zmiennej `x` przechowującej liczbę sekund na liczbę minut, w taki sposób:

```
double duration=3600;
to_minutes_inplace(&duration);
printf("%.2lf", duration); // pokaże 60.00
```

- Napisać funkcję `void cyclic_swap(float* x, float* y, float* z)`, która będzie podmieniać wartości `x,y,z` wg następującej zależności:

```
/*
x->y->z->x
*/
float a=3.45, b=5.63, c=-66.54;
cyclic_swap(&a, &b, &c);
printf("%.3f %.3f %.3f \n",a,b,c); // -66.540 3.450 5.630
cyclic_swap(&a, &b, &c);
printf("%.3f %.3f %.3f \n",a,b,c); // 5.630 -66.540 3.450
cyclic_swap(&a, &b, &c);
printf("%.3f %.3f %.3f \n",a,b,c); // 3.450 5.630 -66.540
```

- Porównanie tablic ze wskaźnikami:

- o zadeklarować tabelę `int tab[10]` wypełnić liczbami od `0` do `9`.
 - o sprawdzić, za pomocą `printf()`, czy adres elementu pierwszego, `&tab[0]` to to samo co `tab` (czy samo `tab` jest wskaźnikiem? jaki adres przechowuje?)
 - o sprawdzić, za pomocą `printf()`, czy wartość na indeksie `0`, `tab[0]`, to jest to samo co `*(tab + 0)` (lub po prostu `*tab`, wartość na którą wskazuje `tab`)
 - o sprawdzić, czy adres elementu `i`-tego tablicy (`i` od `1` do `9`), `&tab[i]` to jest to samo co `tab + i`. (czy `tab+i` to wskaźnik? na adres przechowuje?)
 - o sprawdzić, czy wartość na `i`-tym indeksie, `tab[i]`, to jest to samo co `*(tab + i)` (na jaką wartość wskazuje `tab + i`?)
6. Napisać funkcję `double avg(double* tab, int size)`, która policzy średnią arytmetyczną z elementów tablicy zmiennych typu `double`, podanej do funkcji.
7. Napisać funkcję `void reverse(int* tab, int size)`, która odwróci kolejność elementów w wejściowej tablicy.
8. Napisać funkcję `int GCD(int a, int b)`, która zaimplementuje poniższy pseudokod na algorytm obliczający największy wspólny dzielnik (Greatest Common Divisor) liczb `a, b`:

$\text{GCD}(a, b \in \mathbb{N}) :$

WHILE ($b \neq 0$)

$t := b$

$b := a \bmod b$

$a := t$

RETURN a

9. (rozwińcie 5) Napisać funkcję `void array_means(double* tab, int size, double* ar, double* geom, double* har)`, której zadaniem jest policzenie 3 rodzajów wartości średniej dla elementów tablicy wejściowej `tab`. Wyniki: odpowiednio średnia arytmetyczna, geometryczna i harmoniczna, mają być zapisane pod trzema adresami zmiennych podanymi przez użytkownika: `ar`, `geom`, `har`.

UWAGA: średnia harmoniczna nie jest zdefiniowana dla liczb ≤ 0 , a z kolei geometryczna, dla liczb < 0 . Jeśli nie da się policzyć tych średnich - zapisać pod adresy `geom` bądź `har` wartość `NAN` z modułu `<math.h>`

UWAGA2: jeśli `size` to dodatni `int`, to `1/size` to też `int` (ile równy? ZAWSZE), ale `1.0/size` to już `double`.

1. Arytmetyczna = $\frac{1}{\text{size}} \sum_{i=0}^{\text{size}-1} \text{tab}[i]$

2. Geometryczna = $(\text{tab}[0] \cdot \text{tab}[1] \cdot \text{tab}[2] \dots \text{tab}[\text{size} - 2] \cdot \text{tab}[\text{size} - 1])^{1/\text{size}}$

3. Harmoniczna = $\text{size} / \left(\sum_{i=0}^{\text{size}-1} (1/\text{tab}[i]) \right)$

```
#include <math.h>
/*
(..) reszta programu
*/

//przykład wykorzystania NAN w funkcji
double divide_10_by_a(double a){
    if ( fabs(a) < 1e-30 ) //nie chcemy dzielić przez prawie zero
        return NAN;
    else
        return 10/a;
}
```

Przykładowy, oczekiwany wynik (kod i stdout poniżej - w funkcji warto wyświetlić wiadomość, jeśli nie da się policzyć danej średniej):

```
double numbers[] = {3,5,4,-3,6};
double A, G, H;
array_means(&numbers[0], 5, &A, &G, &H );
printf("ar= %.5lf , geo=%.5lf, har=%.5lf \n", A, G,H );
double num2[] = {1,2,3,5,6};
array_means(&num2[0], 5, &A, &G, &H );
printf("ar= %.5lf , geo=%.5lf, har=%.5lf \n", A, G,H );
```

```
detected negative numbers, cant compute geometric mean.
detected nonpositive numbers, cant compute harmonic mean.
dla tabeli numbers: ar= 3.00000 , geo=nan, har=nan
dla tabeli num2: ar= 3.40000 , geo=2.82523, har=2.27273
```

10. Zaimplementować algorytm sortujący `bubbleSort` jako funkcję `void bubbleSort(int* tab, int size)`, według poniższego pseudokodu:

```
bubbleSort(tab  $\in \mathbb{N}^{\text{size}}$ ) :  
  
  swapped := TRUE  
  
  WHILE(swapped)  
  
    swapped := FALSE  
  
    FOR( $i := 1; i < \text{size}; i := i + 1$ )  
  
      IF(tab[ $i - 1$ ] > tab[ $i$ ])  
  
         $a := \text{tab}[i]$   
  
        tab[ $i$ ] := tab[ $i - 1$ ]  
  
        tab[ $i - 1$ ] :=  $a$   
  
        swapped := TRUE  
  
      ENDIF  
  
    ENDFOR  
  
  ENDWHILE
```

11. Zaimplementować funkcję rekurencyjną `double compound_interest(double v, double r, n)`, która policzy końcowy kapitał wg. procentu składanego, gdzie kapitalizacja następuje raz w roku. V to kapitał początkowy (w zł), r to oprocentowanie roczne (liczba >0 ale <1), a n to liczba lat, po której chcemy sprawdzić wielkość kapitału.

o Wielkość kapitału V_n po n latach to:

- $V_n := V_{n-1}(1 + r)$, dla $n > 0$
- $V_n = V$, dla $n = 0$.

12. **Potęgowanie przez podnoszenie do kwadratu** - generalna metoda szybkiego podnoszenia do dużych potęg całkowitych. Zaimplementować poniższą rekurencję $P(x, n)$ jako funkcję rekurencyjną `double pow_square(double x, int n)`:

- $P(x, n)$ to:
 - 0, gdy $x = 0$,
 - $P(1/x, -n)$, gdy $n < 0$,
 - 1, gdy $n = 0$,
 - $P(x \cdot x, n/2)$, gdy n jest parzyste i $n > 0$,
 - $x \cdot P(x \cdot x, (n - 1)/2)$, gdy n jest nieparzyste i $n > 0$.

13. **Porównanie tempa wzrostu.** (*) Oprócz zwykłej silni liczby n , $n!$, istnieje wiele innych rekurencyjnych funkcji, których wartość rośnie nawet szybciej. W tym zadaniu trzeba porównać tempo wzrostu kilku takich funkcji.

- 2^n
- silnia: $n! := 1 \cdot 2 \cdot \dots \cdot (n - 1) \cdot n$
- "supersilnia": $sf(n) := 1! \cdot 2! \cdot 3! \dots (n - 1)! \cdot n!$
- "hipersilnia": $H(n) := 1^1 \cdot 2^2 \cdot \dots \cdot (n - 1)^{n-1} \cdot n^n$
- "wieża potęg" lub "tetracja"dwójki $2 \uparrow\uparrow n$ to:
 - 1, jeśli $n = 0$
 - $2^{2^{\uparrow\uparrow(n-1)}}$, gdy $n > 0$.
 - np. $2 \uparrow\uparrow 4 = 2^{2^{2^2}}$

Wykorzystując dla zmiennych typ `unsigned long long` (do trzymania dużych liczb całkowitych ≥ 0), napisać rekurencyjne implementacje tych funkcji (można pominąć 2^n).

Potem, przetestować - policzyć wartości dla n od 0 do 8 i porównać, która funkcja rośnie najszybciej, a która najwolniej.

UWAGA: w przypadku tetracji i hipersilni powinniśmy zauważyć coś dziwnego - od pewnego n , wartości wrócą do zera i znowu zaczną rosnać. Wynika to z formatu `unsigned`. Doczytać, dlaczego tak się dzieje i wziąć to pod uwagę w porównaniu.